Baza de date pentru gestiunea unui lanț hotelier

Galoș Ioana
Academia de Studii Economice din București, Facultatea de Cibernetică, Statistică și Informatică Economică
Proiect la disciplina Baze de date

1. Descrierea temei

Am construit o bază de date pentru un lanț de hoteluri din România. Ea ține evidența hotelurilor, a angajaților și a funcțiilor pe care le ocupă, a camerelor, a clienților și a facturilor emise. Pe lângă stocarea datelor, am vrut ca baza de date să poată răspunde la întrebări practice pe care le-ar avea cineva care administrează hotelurile: cât costă salariile în fiecare hotel, ce clienți nu au cerut factură, ce camere sunt ocupate sau cum arată ierarhia angajaților într-un hotel.

Proiectul este realizat în Oracle SQL. Toate datele folosite sunt fictive.

2. Structura bazei de date

Baza de date are șase tabele, legate între ele prin chei primare și chei străine.

Tabelă	Ce conține	Cheie primară	Chei străine
HOTELURI_1	numele, adresa și numărul de stele	id_hotel	–
FUNCTII	denumirea funcției și salariul minim	id_functie	–
AANGAJATI	datele de contact, vechimea, salariul și feedbackul	id_angajat	id_functie, id_hotel
CAMERE	numărul de persoane și prețul pe noapte	id_camera	id_hotel
CLIENTI	datele de contact și camera în care sunt cazați	id_client	id_camera
FACTURI	data, durata șederii și totalul de plată	id_factura	id_client, id_camera

Tabelul 1. Tabelele bazei de date

Fiecare hotel are mai multe camere și mai mulți angajați, iar fiecare angajat are o singură funcție. Un client este cazat într-o cameră și poate primi o factură, care face legătura dintre client și camera respectivă. Pe lângă chei, am adăugat câteva restricții care țin datele corecte: numele hotelului trebuie să fie unic, numărul de stele trebuie să fie între 1 și 5, salariul trebuie să fie pozitiv, iar feedbackul angajatului poate fi doar P (pozitiv) sau N (negativ).

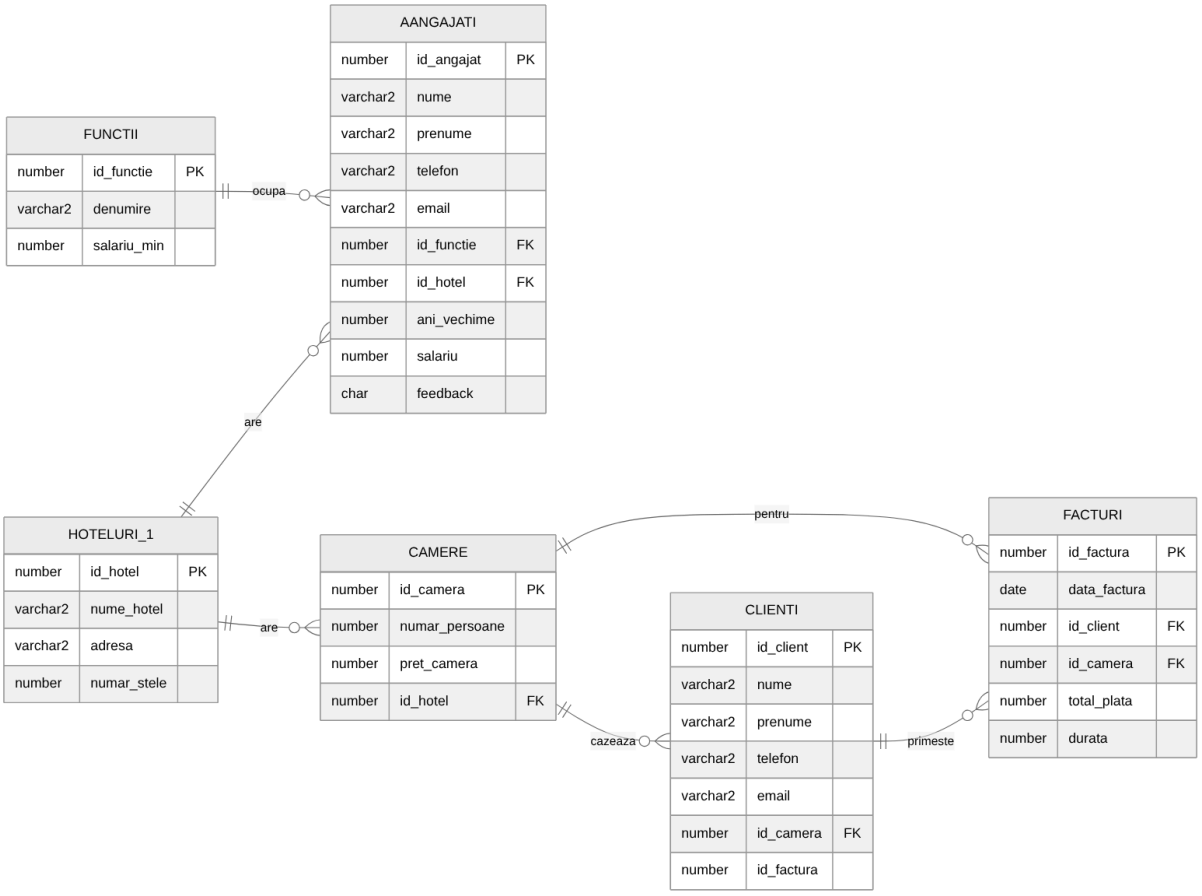


Fig. 1. Schema conceptuală a bazei de date

3. Crearea tabelelor și popularea lor

Am creat tabelele în ordinea în care depind unele de altele: întâi hotelurile și funcțiile, apoi angajații și camerele, la final clienții și facturile. Restricțiile le-am adăugat separat, cu ALTER TABLE.

```
drop table FACTURI cascade constraints;
drop table CLIENTI cascade constraints;
drop table CAMERE cascade constraints;
drop table AANGAJATI cascade constraints;
drop table FUNCTII cascade constraints;
drop table HOTELURI_1 cascade constraints;

create table HOTELURI_1
(
id_hotel NUMBER(3) primary key,
nume_hotel VARCHAR2(50) not null,
adresa VARCHAR2(80),
numar_stele NUMBER(1)
);

alter table HOTELURI_1 add constraint HOTELURI_UK_1 unique(nume_hotel);
alter table HOTELURI_1 add constraint HOTELURI_CK_STELE check (numar_stele between 1 and
5);

create table FUNCTII
(
id_functie NUMBER(3) primary key,
denumire varchar2(30) not null,
```

```
salariu_min number(4)
);

create table AANGAJATI (
id_angajat number(3) primary key,
nume varchar2(20) not null,
prenume varchar2(20),
telefon varchar2(20),
email varchar2(50),
id_functie number(3),
id_hotel number(3),
ani_vechime number(2),
salariu number(4),
feedback char(1)
);

alter table AANGAJATI
add constraint AANGAJATI_FK
foreign key (id_hotel) references HOTELURI_1 (id_hotel);

alter table AANGAJATI
add constraint AANGAJATI_FK2
foreign key (id_functie) references FUNCTII (id_functie);

alter table AANGAJATI add constraint AANGAJATI_CK_SAL check (salariu > 0);
alter table AANGAJATI add constraint AANGAJATI_CK_FEEDBACK check (feedback in ('P',
'N'));

create table CAMERE (
id_camera number(2) primary key,
numar_persoane number(1),
pret_camera number(4),
id_hotel number(3)
);

alter table CAMERE
add constraint CAMERE_FK_HOTEL
foreign key (id_hotel) references HOTELURI_1 (id_hotel);

create table CLIENTI (
id_client number(3) primary key,
nume varchar2(20) not null,
prenume varchar2(20),
telefon varchar2(20),
email varchar2(50),
id_camera number(2),
id_factura number(2)
);

alter table CLIENTI
add constraint CLIENTI_FK_CAMERA
foreign key (id_camera) references CAMERE (id_camera);

create table FACTURI
(
id_factura number(2) primary key,
data_factura date,
id_client number(3),
id_camera number(2),
total_plata number(5),
durata number(2)
);
```

```
alter table FACTURI add constraint FACTURI_FK1
foreign key (id_client) references CLIENTI (id_client);

alter table FACTURI add constraint FACTURI_FK2
foreign key (id_camera) references CAMERE (id_camera);
```

Am introdus apoi 10 hoteluri, 10 funcții, 16 angajați, 14 camere, 14 clienți și 10 facturi. Totalul fiecărei facturi este prețul camerei înmulțit cu numărul de nopți, iar salariul fiecărui angajat respectă salariul minim al funcției lui. Scriptul complet cu datele se găsește în fișierul data.sql.

4. Actualizarea și ștergerea datelor

Am modificat câte o înregistrare din fiecare tabelă și am afișat tabela după fiecare modificare, ca să se vadă efectul. Apoi am șters tabela de angajați și am recuperat-o cu FLASHBACK, iar la final am șters un client care nu avea factură, pentru că un client cu factură nu poate fi șters cât timp factura face referire la el.

Schimbarea prenumelui unui angajat, a prețului unei camere, a prenumelui unui client, a totalului unei facturi, a denumirii unei funcții și a numelui unui hotel:

```
UPDATE AANGAJATI
SET prenume='ANDREI' where id_angajat=101;
select * from aangajati;

update CAMERE
set pret_camera=260 where id_camera=14;
select * from camere;

UPDATE CLIENTI
SET prenume='VASILE' where id_client=410;
select * from clienti;

UPDATE FACTURI
SET total_plata=550 where id_factura=6;
select * from facturi;

update functii
set denumire='Manager general' where id_functie=100;
select * from functii;

update hoteluri_1
set nume_hotel='NOXUS' where id_hotel=950;
select * from hoteluri_1;
```

Ștergerea și recuperarea tabelei AANGAJATI:

```
Drop Table aangajati;
Flashback Table aangajati TO BEFORE DROP;
```

Ștergerea unui client fără factură:

```
DELETE FROM CLIENTI
WHERE id_client = 320;
```

5. Joncțiuni

Joncțiunile leagă informațiile din mai multe tabele. Am folosit toate tipurile, iar diferența dintre ele se vede cel mai bine la clienți și facturi: unii clienți nu au cerut factură, deci nu au

corespondent în tabela FACTURI.

Facturile împreună cu clientul fiecăreia (RIGHT JOIN) și toate combinațiile, inclusiv cele fără corespondent (FULL OUTER JOIN):

```
SELECT c.nume, f.id_factura
FROM clienti c
RIGHT JOIN facturi f ON c.id_client = f.id_client;

SELECT c.nume, f.id_factura
FROM clienti c
FULL OUTER JOIN facturi f ON c.id_client = f.id_client;
```

Angajații cu salariul lor și hotelul la care lucrează:

```
SELECT A.nume AS Nume_Angajat, A.salariu AS Salariu, H.nume_hotel AS Nume_Hotel,
H.adresa AS Adresa_Hotel
FROM AANGAJATI A
JOIN HOTELURI_1 H ON A.id_hotel = H.id_hotel;
```

Angajații cu feedback pozitiv și hotelul la care lucrează:

```
select aa.nume, aa.prenume, h.nume_hotel, aa.feedback from aangajati aa inner join
hoteluri_1 h
on aa.id_hotel = h.id_hotel where aa.feedback = 'P';
```

Toți clienții, cu sau fără factură, și apoi doar cei care nu au cerut factură:

```
select c.nume, c.prenume, f.id_factura from clienti c left join facturi f on
c.id_factura = f.id_factura;

select c.nume, c.prenume, f.id_factura from clienti c left join facturi f on
c.id_factura = f.id_factura where c.id_factura is null;
```

6. Funcții de grup

Funcțiile de grup calculează un singur rezultat pentru un set de rânduri. Le-am folosit mai ales pe salarii, unde au cel mai mult sens: totalul pe hotel, minimul, media și numărul de valori distincte.

Totalul salariilor pe fiecare hotel, salariul minim și numărul de clienți din fiecare cameră:

```
SELECT id_hotel, SUM(salariu)
FROM aangajati
GROUP BY id_hotel;

SELECT MIN(salariu)
FROM aangajati;

SELECT id_camera, COUNT(*)
FROM clienti
GROUP BY id_camera;
```

Salariul mediu și numărul de salarii distincte:

```
SELECT ROUND(AVG(salariu), 2) AS Salariu_Mediu
FROM AANGAJATI;

SELECT count(distinct salariu) FROM aangajati;
```

Hotelurile în care salariul mediu depășește 4000:

```
SELECT id_hotel, ROUND(AVG(salariu)) AS Salariu_Mediu
FROM aangajati
```

```
GROUP BY id_hotel
HAVING AVG(salariu) > 4000;
```

Clienții care au de plătit între 250 și 400 de lei, de la suma cea mai mare la cea mai mică:

```
select c.nume, c.prenume, f.total_plata
from clienti c, facturi f
where c.id_factura = f.id_factura and f.total_plata between 250 and 400
order by f.total_plata desc;
```

7. Interogări ierarhice

Pentru partea ierarhică am adăugat în tabela AANGAJATI coloana id_sef. Managerul fiecărui hotel nu are șef, iar ceilalți angajați din hotel îi sunt subordonați. Cu CONNECT BY am putut afișa apoi structura pe niveluri, subordonații unui anumit manager, nivelul maxim din ierarhie și traseul complet de la manager la fiecare angajat.

Adăugarea coloanei și completarea ei:

```
alter table AANGAJATI add id_sef number(3);

update AANGAJATI set id_sef = null where id_angajat in (101,201,301,401);
update AANGAJATI set id_sef = 101 where id_angajat in (102,103,104,105);
update AANGAJATI set id_sef = 201 where id_angajat in (202,203);
update AANGAJATI set id_sef = 301 where id_angajat in (302,303,304,305,306);
update AANGAJATI set id_sef = 401 where id_angajat in (402);
```

Ierarhia tuturor angajaților:

```
select level as nivel,
id_angajat,
nume,
prenume
from AANGAJATI
start with id_sef is null
connect by prior id_angajat = id_sef;
```

Subordonații managerului cu id-ul 101:

```
SELECT LEVEL AS nivel,
id_angajat,
nume,
prenume
FROM AANGAJATI
START WITH id_angajat = 101
CONNECT BY PRIOR id_angajat = id_sef;
```

Ierarhia împreună cu hotelul la care lucrează fiecare angajat:

```
SELECT LEVEL AS nivel,
a.id_angajat,
a.nume,
a.prenume,
h.nume_hotel
FROM AANGAJATI a
JOIN HOTELURI_1 h ON a.id_hotel = h.id_hotel
START WITH a.id_sef IS NULL
CONNECT BY PRIOR a.id_angajat = a.id_sef;
```

Nivelul ierarhic maxim și traseul complet:

```
SELECT MAX(LEVEL) AS nivel_maxim
FROM AANGAJATI
START WITH id_sef IS NULL
CONNECT BY PRIOR id_angajat = id_sef;
```

```
SELECT SYS_CONNECT_BY_PATH(nume, ' -> ') AS traseu_ierarhic
FROM AANGAJATI
START WITH id_sef IS NULL
CONNECT BY PRIOR id_angajat = id_sef;
```

8. Operatori pe mulțimi

UNION, INTERSECT și MINUS compară rezultatele a două interogări. UNION adună numele angajaților și ale clienților fără dubluri, INTERSECT arată camerele în care există clienți cazați, iar MINUS arată angajații al căror nume nu apare și printre clienți.

```
SELECT nume FROM aangajati
UNION
SELECT nume FROM clienti;

SELECT id_camera FROM camere
INTERSECT
SELECT id_camera FROM clienti;

SELECT nume FROM aangajati
MINUS
SELECT nume FROM clienti;
```

9. DECODE și CASE

Ambele întorc o valoare diferită în funcție de o condiție. DECODE e mai potrivit pentru valori fixe, de exemplu numărul de stele transformat în simboluri. CASE e mai flexibil și merge și pentru intervale, cum e clasificarea angajaților după salariu sau după vechime.

Un bonus acordat unui singur angajat și stelele fiecărui hotel:

```
SELECT
nume,
prenume,
DECODE(nume, 'MARINESCU', 500, 0) AS Bonus
FROM aangajati;

SELECT nume_hotel, DECODE(numar_stele, 1, '*', 2, '**', 3, '***', 4, '****', 5, '*****')
stele FROM hoteluri_1;
```

Clasificarea după salariu și după vechime:

```
SELECT
ID_angajat,
nume,
salariu,
CASE
WHEN SALARIU >= 2500 THEN 'Mare'
ELSE 'Mic'
END AS STARE_SALARIU
FROM aangajati;

SELECT
id_angajat,
nume,
ani_vechime,
CASE
WHEN ani_vechime >= 5 THEN 'Experimentat'
WHEN ani_vechime >= 2 THEN 'Intermediar'
ELSE 'Incepator'
```

```
END AS Nivel_Experienta
FROM aangajati;
```

10. Cereri imbricate

O cerere imbricată folosește rezultatul unei alte interogări. În primul exemplu subinterogarea numără angajații pentru fiecare hotel în parte, iar în al doilea exclude clienții cazați la hotelul cu id-ul 300.

```
SELECT H.nume_hotel,
(SELECT COUNT(*)
FROM AANGAJATI A
WHERE A.id_hotel = H.id_hotel) AS Numar_Angajati
FROM HOTELURI_1 H;

select c.id_client, c.nume, c.prenume
from clienti c
where c.id_client not in (
select c2.id_client
from clienti c2
join camere ca2 on ca2.id_camera = c2.id_camera
where ca2.id_hotel = 300
);
```

11. Funcții single-row

Aceste funcții se aplică pe fiecare rând în parte. Le-am folosit pentru text (concatenare, majuscule, inițiale) și pentru date calendaristice (anul emiterii unei facturi, un interval de date).

Funcții pe text:

```
SELECT nume || ' ' || prenume AS Nume_Complet
FROM AANGAJATI;

SELECT UPPER(nume_hotel) AS Nume_Hotel_Majuscule
FROM HOTELURI_1;

select substr(nume, 1, 1) as Initiala_nume, substr(prenume, 1, 1) as Initiala_prenume
from aangajati;
```

Funcții pe date și filtrare după o listă de valori:

```
select id_factura, data_factura from facturi where
EXTRACT(YEAR FROM data_factura) != 2022;

SELECT id_factura, data_factura FROM facturi
WHERE data_factura BETWEEN TO_DATE('2022-01-15','yyyy-mm-dd') AND TO_DATE('2022-01-
31','yyyy-mm-dd');

SELECT nume, prenume FROM clienti WHERE id_client IN (110,330,420);
```

12. Alte obiecte ale bazei de date

La final am creat o vizualizare cu angajații și hotelurile lor, un sinonim pentru tabela CLIENTI (pe care l-am șters apoi), un index pe totalul facturilor, ca interogările după sumă să fie mai rapide, și o secvență care poate genera id-uri noi pentru clienți, începând de la 1000.

```
CREATE OR REPLACE VIEW Vizualizare_Angajati_Hoteluri AS
SELECT A.nume AS Nume_Angajat, H.nume_hotel AS Nume_Hotel
FROM AANGAJATI A
```

```
JOIN HOTELURI_1 H ON A.id_hotel = H.id_hotel;

SELECT * FROM Vizualizare_Angajati_Hoteluri;

CREATE SYNONYM cc FOR clienti;
SELECT * FROM cc;
DROP SYNONYM cc;

CREATE INDEX idx_tot_plata ON facturi(total_plata);

CREATE SEQUENCE seq_id_client
start with 1000
increment by 1
NOCACHE
NOCYCLE;

select *
from user_sequences
where sequence_name = 'SEQ_ID_CLIENT';
```